Digital Systems

Lecture 5 (Part I)

Optimising Assembly

Mark van der Wilk

Hilary Term 2026

Department of COMPUTER SCIENCE
UNIVERSITY OF OXFORD

Following the course of past years by Mike Spivey

Recap: Last time

ⓘ Understanding Compilation Process

What is specified after compilation, and what after linking.

💪 Programming in Assembly

- Assembly has quirks: They are there for a reason! (Remember?)
- Looking them up in references.

💪 Using a Debugger

- Stepping through assembly, instruction-by-instruction.
- Viewing effect on registers.

Naive Multiplication in C

```
unsigned func(unsigned a, unsigned b) {
    unsigned x = a, y = b, z = 0;

    /* Invariant: a × b = x × y + z */
    while (y != 0) {
        y = y - 1;
        z = z + x;
    }

    return z;
}
```

Which Instructions are Actually Used?

```
$ arm-none-eabi-objdump -d mul1.o
00000000 <func>:
   0:  2200      movs    r2, #0

00000002 <loop>:
   2:  2900      cmp     r1, #0
   4:  d002      beq.n   0xc <done>
   6:  3901      subs    r1, #1
   8:  1812      adds    r2, r2, r0
   a:  e7fa      b.n     0x2 <loop>

0000000c <done>:
   c:  0010      movs    r0, r2
   e:  4770      bx      lr
```

Look at the Brach Offset

Encoding T1 All versions of the Thumb instruction set.

```
B<c>  <label>
```

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	cond				imm8							

Encoding T2 All versions of the Thumb instruction set.

```
B<c> <label>
```

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	imm11										

Pipelining

? Why is there an offset of 4 in relative addresses in jumps?

- `4: beq.n` would jump +2 instructions ahead.
- `a: b.n` would jump −6 instructions ahead.

Pipelining

? Why is there an offset of 4 in relative addresses in jumps?

- `4: beq.n` would jump +2 instructions ahead.
- `a: b.n` would jump −6 instructions ahead.

Executing an instruction is more complex than previously described.

- Executing instruction is broken up into stages.
- In ARM Cortex-M0, 3 cycles needed to execute a single instruction.
- However, stages operate in *parallel*, so still 1 instruction *per* cycle.

Pipelining: Branch offset

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch executed now!

-
-
-
-

Pipelining: Branch offset

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch executed now!

- When branch is executed, `pc` is 8.
-
-
-

Pipelining: Branch offset

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch executed now!

- When branch is executed, `pc` is 8.
- Want to jump two instructions ahead.
-
-

Pipelining: Branch offset

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch executed now!

- When branch is executed, `pc` is 8.
- Want to jump two instructions ahead.
- $\Rightarrow$ Encode offset of 2 instructions!
-

Pipelining: Branch offset

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch executed now!

- When branch is executed, `pc` is 8.
- Want to jump two instructions ahead.
- $\Rightarrow$ Encode offset of 2 instructions!
- CPU handles multiplication by two (bit shift) before adding to `pc`.

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

-
-
-

-
-

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

-
-
-
-
-

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

- Usually: One cycle per instruction.
-
-
-
-

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

- Usually: One cycle per instruction.
- Branch: Taken branch takes 2 cycles *extra* (3 total), otherwise 1.
-
-
-

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

- Usually: One cycle per instruction.
- Branch: Taken branch takes 2 cycles *extra* (3 total), otherwise 1.
- Load / store: Two cycles (one extra for memory access).

-
-

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

- Usually: One cycle per instruction.
- Branch: Taken branch takes 2 cycles *extra* (3 total), otherwise 1.
- Load / store: Two cycles (one extra for memory access).

Find number of cycles:

-
-

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

- Usually: One cycle per instruction.
- Branch: Taken branch takes 2 cycles *extra* (3 total), otherwise 1.
- Load / store: Two cycles (one extra for memory access).

Find number of cycles:

- Can measure (oscilloscope, see labs)
-

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

- Usually: One cycle per instruction.
- Branch: Taken branch takes 2 cycles *extra* (3 total), otherwise 1.
- Load / store: Two cycles (one extra for memory access).

Find number of cycles:

- Can measure (oscilloscope, see labs)
- Count instructions (7 cycles)

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

- Usually: One cycle per instruction.
- Branch: Taken branch takes 2 cycles *extra* (3 total), otherwise 1.
- Load / store: Two cycles (one extra for memory access).

Find number of cycles:

- Can measure (oscilloscope, see labs)
- Count instructions (7 cycles)

? Why do branches take 3 cycles?

Pipelining: Normal Operation

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch not taken
6	10	`b`	`adds`	`subs`	next instruction as normal
7	12	`movs`	`b`	`adds`	

Pipelining: Branch Taken

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch taken!
6	12	`movs`	`adds X`	`subs X`	do not execute!
7	14	`bx`	`movs`	`adds X`	do not execute!
8	16	`...`	`bx`	`movs`	as normal

Can We Do Better?

Solutions:

- Better algorithm (e.g. $O(\log n)$).
- "Squeeze" loop.

Optimisation

```
loop:
    cmp r1, #0          @ if y == 0
    beq done            @     jump to done
loop1:
    subs r1, r1, #1     @ y = y - 1
    adds r2, r2, r0     @ z = z + x
    cmp r1, #0          @ if y == 0
    beq done            @     jump to done
    b loop1             @ jump to loop1
done:
```

Optimisation

Swap and simplify.

```
loop:
    cmp r1, #0          @ if y == 0
    beq done            @     jump to done
loop1:
    adds r2, r2, r0     @ z = z + x
    subs r1, r1, #1     @ y = y - 1
    cmp r1, #0          @ if y != 0
    bne loop1           @     jump to loop1
done:
```

Optimisation

Let subs set the condition codes.

```
loop:
    cmp r1, #0              @ if y == 0
    beq done                @     jump to done
loop1:
    adds r2, r2, r0         @ z = z + x
    subs r1, r1, #1         @ y = y - 1
    bne loop1               @ if y != 0, loop
done:
```

- 5 cycles per iteration!
- Would be `do..while` loop in C.

Loop Unrolling

Duplicate the loop body n times. m

```
loop1:
    adds r2, r2, r0
    subs r1, r1, #1
    beq done
    adds r2, r2, r0
    subs r1, r1, #1
    beq done
    ...
```

```
    adds r2, r2, r0
    subs r1, r1, #1
    beq done
    adds r2, r2, r0
    subs r1, r1, #1
    bne loop1
done:
```

- 3.5 cycles per iteration!

Observations on Assembly Programming

- Writing optimised asm code is labourious! Small changes in details of program impact many other lines (e.g. change in register!)
-
-

Observations on Assembly Programming

- Writing optimised asm code is labourious! Small changes in details of program impact many other lines (e.g. change in register!)
- Good compilers can generate very efficient assembly code from C code.
  - E.g.: It's hard for us to use higher registers. Requires knowing a lot of these quirks of assembly, which instructions can use high registers, and how. Compilers can do this well.
-

Observations on Assembly Programming

- Writing optimised asm code is labourious! Small changes in details of program impact many other lines (e.g. change in register!)
- Good compilers can generate very efficient assembly code from C code.
  - E.g.: It's hard for us to use higher registers. Requires knowing a lot of these quirks of assembly, which instructions can use high registers, and how. Compilers can do this well.
- It's good to know *how* to write a very fast, tight loop, but no need to spend all your time doing this for *every* bit of code. (Amdahl's law)

Observations on Assembly Programming

- Writing optimised asm code is labourious! Small changes in details of program impact many other lines (e.g. change in register!)
- Good compilers can generate very efficient assembly code from C code.
  - E.g.: It's hard for us to use higher registers. Requires knowing a lot of these quirks of assembly, which instructions can use high registers, and how. Compilers can do this well.
- It's good to know *how* to write a very fast, tight loop, but no need to spend all your time doing this for *every* bit of code. (Amdahl's law)

Debugger is *super* helpful for writing code. Learn it!

Recap

💡 Pipelining

- Instructions are executed over multiple clock cycles in parallel.
- Complicates branch instructions, and makes them slower.

💪 Determining execution time of assembly program

💪 More complex assembly programming, and optimisation

Digital Systems

Lecture 5 (Part II)

Subroutines

Mark van der Wilk

Hilary Term 2026

UNIVERSITY OF OXFORD

Department of COMPUTER SCIENCE

Following the course of past years by Mike Spivey

Previously: We created a subroutine

Problems:

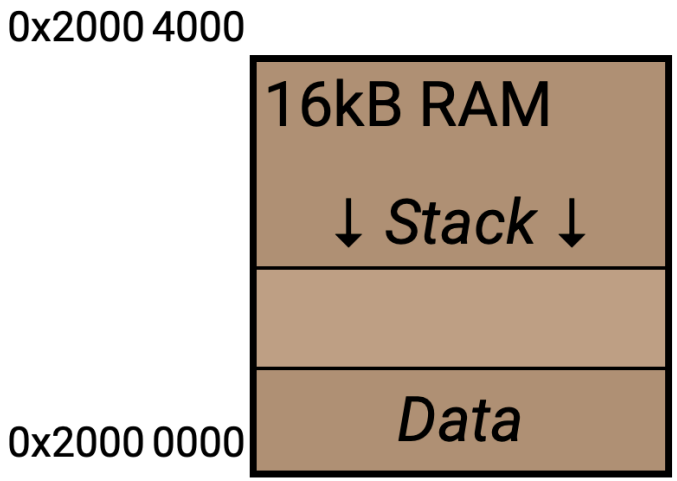
- Could not call other subroutines.
- Could only use `r0` - `r3`, while leaving `r4` and above unchanged.

Use a *Stack* for local variables

- Let subroutines use memory relative to the *stack pointer* (register `sp`).
- Memory below in the stack is used by calling subroutines, and should remain untouched.
- Memory above in the stack is free to be used by the subroutine.

This is only one possible solution, but is the one chosen by design in ARM. Special instructions are created to facilitate using the stack.

The Stack



Example:

`main()`
`sub1()`
`sub2()`
`sub3()`

Return:

`main()`
`sub1()`
`sub2()`

Call next:

`main()`
`sub1()`
`sub2()`
`sub4()`

Call next:

`main()`
`sub1()`
`sub2()`
`sub4()`
`sub5()`

- Convention: Stack starts at top of address space, and grows downwards. So "above" in the stack has a lower address!
- Each time a subroutine is called, all memory below `sp` is a usable region of memory (stack frame), that won't interfere with other subroutines.
- When a subroutine returns, the caller finds its stack frame as it left it.

Subroutine Contract

Calling function:

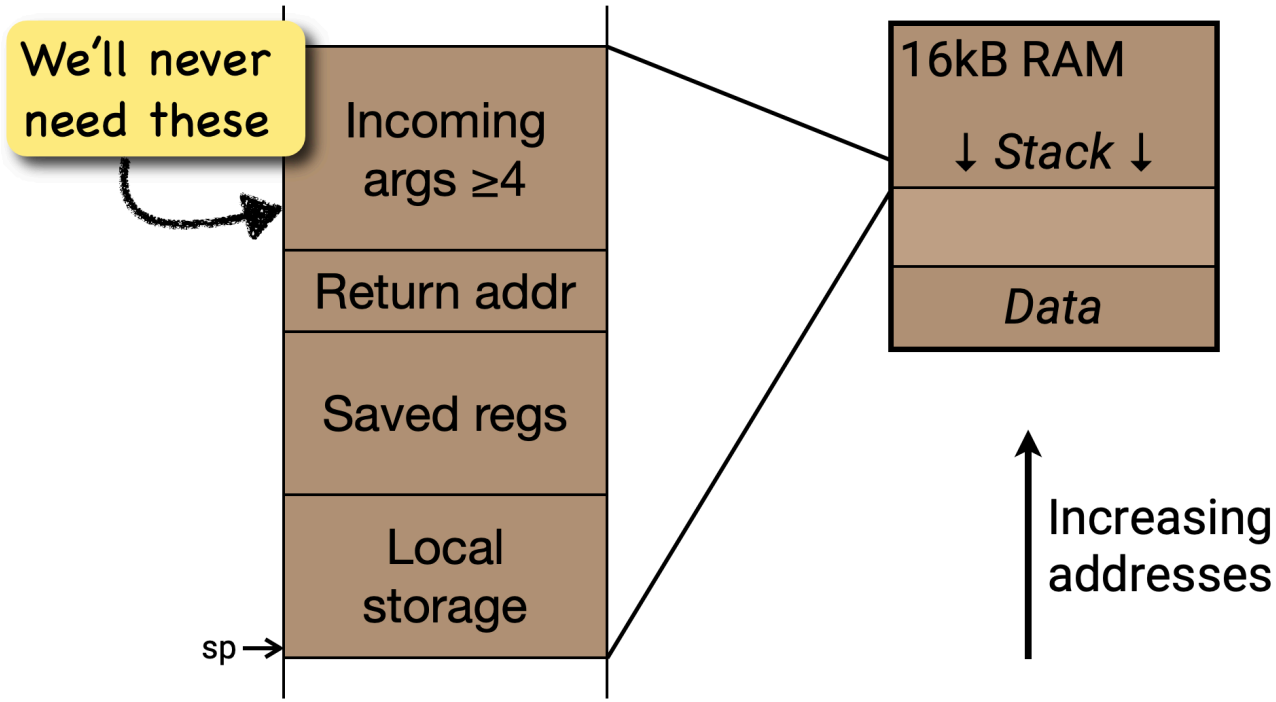
- Places first arguments 1 … 4 in `r0` - `r3`
- Places arguments ≥ 4 on stack (called function knows how many to expect) (labs won't need this)

Called function:

- Leave values of `r4` and up unchanged on return
- Leave stack pointer `sp` unchanged on return
- Leave stack below current location (i.e. memory addresses above `sp`) unchanged
- Leave return value in `r0`

Subroutine Convention

Subroutine can use stack in whatever way it wants (assuming there is space!), but ARM provides instructions for:



Storing Registers

- `push {REGISTER_LIST}`: Place registers in list on stack in order, and decrease `sp` accordingly.
- `pop {REGISTER_LIST}`: Take values from stack, and place into the registers in list order, and increase `sp` accordingly.

Push & Pop Peculiarities

- `push`/`pop` can store/restore lower registers (`r0 - r7`)
- `push` can store `lr`, as well
- `pop` can restore to `pc`, as well

Prev, we saw we could return from a subroutine using `bx lr`. Using `pop` to restore to `pc` gives the same effect, but without first restoring to `lr`!

> **`push`/`pop` depart from RISC principles!**
>
> Normally, one instruction does one thing, but this gives speedup! Load/store operation is 2 cycles, while `push`/`pop` is 1+n cycles.

Remember, instruction set designers optimised the common case.

Putting it all together

Example: Factorial

```
int fac(int n) {
    int k = n, f = 1;
    while (k != 0) {
        f = mult(f, k);
        k = k-1;
    }
    return f;
}
```

Subroutine calls another subroutine! So we will use what we discussed.

Example: Factorial, opening

- As usual, let's start by converting code to assembly, and not worrying about efficiency.
- For clarity, I recommend making clear which variables are assigned to which registers (little table in your exam answer helps!)
- Let's assign `k` to `r4`, and `f` to `r5` (logic behind this?)
- Must use `push`.

```
fac:
    push {r4, r5, lr}
    movs r4, r0                 @ Initialise k to n
    movs r5, #1                 @ Set f to 1
```

Example: Factorial, Loop

Again, we can worry about optimisations later:

```
fac:
    push {r4, r5, lr}
    movs r4, r0                 @ Initialise k to n
    movs r5, #1                 @ Set f to 1
again:
    cmp r4, #0                  @ Is k = 0?
    beq finish                  @ If so, finished
```

Example: Factorial, Subroutine Call

Again, we can worry about optimisations later:

```
...
again:
    cmp r4, #0                  @ Is k = 0?
    beq finish                  @ If so, finished

    movs r0, r5                 @ Set f to f * k
    movs r1, r4
    bl mult                     @ branch link instruction!
    movs r5, r0
```

Example: Factorial, Finish Loop

```
...
again:
    cmp r4, #0                  @ Is k = 0?
    beq finish                  @ If so, finished

    movs r0, r5                 @ Set f to f * k
    movs r1, r4
    bl mult                     @ branch link instruction!
    movs r5, r0

    subs r4, r4, #1             @ Decrement k
    b again                     @ And repeat
```

Example: Factorial, Return Subroutine

```
...
again:
    cmp r4, #0                  @ Is k = 0?
    beq finish                  @ If so, finished

...

finish:
    movs r0, r5                 @ Return f
    pop {r4, r5, pc}
```

Example: Factorial, All Together

```
fac:
    push {r4, r5, lr}
    movs r4, r0                 @ Initialise k to n
    movs r5, #1                 @ Set f to 1
again:
    cmp r4, #0                  @ Is k = 0?
    beq finish                  @ If so, finished

    movs r0, r5                 @ Set f to f * k
    movs r1, r4
    bl mult                     @ branch link instruction!
    movs r5, r0
```

```
    subs r4, r4, #1          @ Decrement k
    b again                  @ And repeat
finish:
    movs r0, r5              @ Return f
    pop {r4, r5, pc}
```

Example: Factorial, optimisation

You may have spotted that `f` can live in `r0` constantly, since the return value of `mult()` will also be the first argument for the next call.

Optimising compilers can spot this.

Summary

How can we create subroutines which can call others?

- Remember value of `lr` that will get overwritten by next jump.

How can we use `r4` and above?

- Must restore of `r4` and above at the end of subroutine.

- See notes for:
  - Expanded discussion
  - More efficient multiplication routine
- Don't forget rainbow chart and instruction overview!